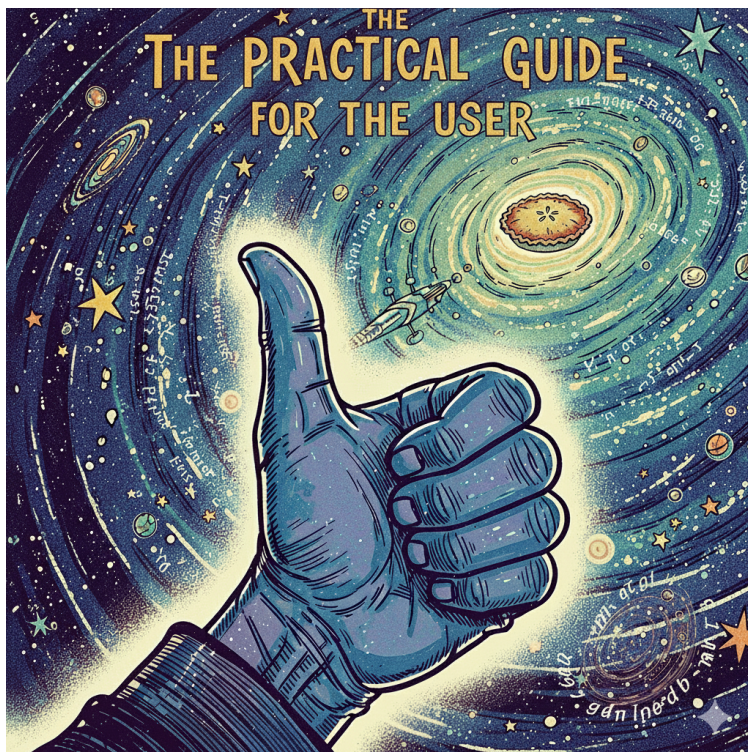


PrimePie: Number Theory Library

Practical User Guide

Jonatan Beiruty

24.7.2025



Contents

1	Introduction	2
2	Algebra	3
2.1	Groups	3
2.2	Rings	4
A	Installation	7

Chapter 1

Introduction

This user guide provides a practical overview of how to use the PrimePie library. Rather than focusing on the underlying mathematical theory or algorithmic design, this document is dedicated to demonstrating how to work with the library from the user's perspective.

Through concise and focused examples, users will learn how to import and interact with the various components of the library, including core number theory utilities, algebraic structures, and cryptographic primitives (**in the future**). Each section includes annotated code snippets and explanations to help both beginners and advanced users integrate PrimePie into their own projects.

For the theoretical background and detailed explanation of the algorithms implemented, please refer to the accompanying document.

Chapter 2

Algebra

The algebra module is organized around two kinds of structures:

1. **Groups**
2. **Rings**

Most algebra operations accept plain Python integers as inputs when this is unambiguous, but they return `Element` objects. An element remembers the group or ring it belongs to, prints in algebraic notation, and exposes its raw Python value through `.value`.

For integer-valued elements, the C++ backend uses GMP's `mpz_class`. This means that algebraic elements are represented with arbitrary-precision integers rather than fixed-width `int64_t` arithmetic.

Python

```
from primepie.algebra import AdditiveModGroup

G = AdditiveModGroup(7)
a = G(10)

print(a)          # 3 mod 7
print(a.value)    # 3
```

2.1 Groups

A group provides an identity element, inverses, a binary operation, and exponentiation by repeated use of the group operation. In an additive group, `power(a, n)` means adding `a` to itself `n` times.

Additive groups modulo `n`.

Python

```
from primepie.algebra import AdditiveModGroup

G = AdditiveModGroup(7)

print(G.identity())    # 0 mod 7
print(G.inverse(3))    # 4 mod 7
```

```
print(G.operate(3, 6))    # 2 mod 7
print(G.power(2, 3))     # 6 mod 7
```

Batch exponentiation.

Python

```
G = AdditiveModGroup(7)

same_exponent = G.power([1, 2, 3], 4)
different_exponents = G.power([1, 2, 3], [0, 1, 2])

print([x.value for x in same_exponent])    # [4, 1, 5]
print([x.value for x in different_exponents]) # [0, 2, 6]
```

Multiplicative groups modulo n . The multiplicative group modulo n contains only the invertible residue classes, that is, elements coprime to n .

Python

```
from primepie.algebra import MultiplicativeModGroup

U = MultiplicativeModGroup(7)

print(U.identity())    # 1 mod 7
print(U.inverse(3))    # 5 mod 7
print(U.operate(3, 5)) # 1 mod 7
print(U.power(3, 4))   # 4 mod 7
```

Direct sums of groups.

Python

```
from primepie.algebra import AdditiveModGroup, DirectSumGroup

Z5 = AdditiveModGroup(5)
Z7 = AdditiveModGroup(7)
G = DirectSumGroup([Z5, Z7])

a = G((2, 3))
b = G((4, 6))

print(a)          # (2 mod 5, 3 mod 7)
print(G.operate(a, b)) # (1 mod 5, 2 mod 7)
print(G.power(a, 3))  # (1 mod 5, 2 mod 7)
```

2.2 Rings

A ring in PrimePie is also used as an additive group. This means `power(a, n)` is additive power, while `mpower(a, n)` is multiplicative power.

Rings modulo n.

Python

```
from primepie.algebra import IntegersModRing

R = IntegersModRing(7)

print(R.zero())      # 0 mod 7
print(R.one())       # 1 mod 7
print(R.add(5, 6))   # 4 mod 7
print(R.mul(3, 5))   # 1 mod 7
print(R.is_equal(10, 3)) # True
```

Additive and multiplicative power.

Python

```
R = IntegersModRing(7)

print(R.power(2, 3))      # 6 mod 7, additive: 2 + 2 + 2
print(R.mpower(3, 4))     # 4 mod 7, multiplicative: 3^4

out = R.mpower([1, 2, 3, 6], 3)
print([x.value for x in out]) # [1, 1, 6, 6]
```

The ring of integers.

Python

```
from primepie.algebra import Integers

Z = Integers()

print(Z.add(5, 6))      # 11
print(Z.mul(5, 6))      # 30
print(Integers.gcd(84, 30)) # 6
print(Integers.is_prime(97)) # True
print(Integers.is_prime([2, 4, 5, 9]))
# [True, False, True, False]
```

Polynomial rings.

Python

```
from primepie.algebra import IntegersModRing, PolynomialRing

R = IntegersModRing(5)
P = PolynomialRing(R)

f = P([1, 2])  # 1 + 2X
g = P([4, 1])  # 4 + X

print(f)      # (1 mod 5) + (2 mod 5) * X
```

```
print(P.add(f, g)) #  $(3 \bmod 5) * X$ 
print(P.mul(f, g)) #  $(4 \bmod 5) + (4 \bmod 5) * X + (2 \bmod 5) * X^2$ 
print(P.add(f, 3)) #  $(4 \bmod 5) + (2 \bmod 5) * X$ 
```

Appendix A

Installation

TODO